**Ex. No. :1 : Experimental verification of Logic gates**
**Date :**

**AIM:**

To study about logic gates and verify their truth tables**.**

**APPARATUS REQUIRED:**

| Sl.No | Components | Specification | Quantity |
|---|---|---|---|
| 1. | NI- Multisim | Version 14.1 | - |
| 2. | PCS | Windows 10 | 1 |

**THOERY:**

- Logic gates are the basic elements that make up a digital system. The gate is a digital circuit with one or more inputs, but only one output. By connecting the different gates in different ways, we can build circuits that perform arithmetic and other functions.
- The operation of a logic gate can be easily understood with the help of "truth table". A truth table is a table that shows all the input-output possibilities of a logic circuit ie., the truth table indicates the outputs for different possibilities of the inputs.
- The types of gates available are the AND, OR, NOT, NAND, NOR, exclusive-OR and the exclusive-NOR. Except for the exclusive-NOR gate they are available in monolithic integrated form.

***AND gate:***

- The AND gates has two or more inputs. It performs a logical multiplication. The output is HIGH (1), when both the inputs are 1; otherwise the output from the gate is LOW (0). The output from the AND gate is written as **A.B**.

***OR gate:***

- The OR gates has two or more inputs. It performs a logical addition. The output is HIGH (1), if any of the inputs are 1; the output is LOW (0) if and only if all the inputs are 0. The output from the AND gate is written as **A+B**.

***NOT gate:***

- The NOT gate has only one input. It performs a basic logic function called inversion. The output is HIGH (1), when the input is 0; the output is LOW (0) when the input is 1. The output from the NOT gate is written as **A'**.

***NAND gate:***

- The NAND gate is a contraction of AND-NOT. It has two or more inputs. The output is HIGH (1), when any of the inputs are 0; the output is LOW (0), if and only if all the inputs are 1. The output from the AND gate is written as (**A.B)'**. It is a universal gate.

***NOR gate:***

- The NOR gate is a contraction of OR-NOT. It has two or more inputs. The output is HIGH (1), when all inputs are 0; the output is LOW (0), when any of the inputs are 1. The output from the AND gate is written as (**A+B)'**. It is a universal gate.

***EX-OR gate:***

- The EX-OR gate has two or more inputs. The output is HIGH (1), when odd number of inputs is 1. The output from the AND gate is written as (**A⊕B).**

**PROCEDURE:**

1. Connections are given as per the logic diagram.
2. Logic inputs are given as per the truth table.
3. Observe the logic output and verify with the truth table.

**i. OR gate:**

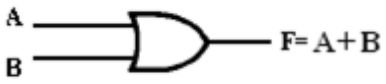



Figure 1.1 Symbol of OR gate

Table 1.1 Truth Table of OR gate

| A | B | F= A+B |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 1 |
| 1 | 0 | 1 |
| 1 | 1 | 1 |

**ii. NOT gate:**

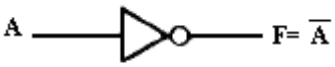



Figure 1.3 Symbol of NOT gate

Table 1.2 Truth Table of NOT gate

| A | F= $\overline{A}$ |
|---|---|
| 0 | 1 |
| 1 | 0 |

**iii. AND gate:**




Figure 1.4 Symbol of AND gate

Table 1.3 Truth Table of AND gate

| A | B | F= A.B |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 0 |
| 1 | 0 | 0 |
| 1 | 1 | 1 |

**iv. 3-Input AND gate:**

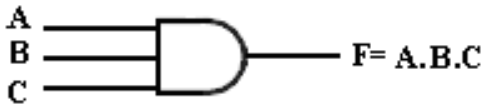



Figure 1.6 Symbol of 3-Input AND gate

Table 1.4 Truth Table of 3-Input AND gate

| A | B | C | F= A.B.C |
|---|---|---|---|
| 0 | 0 | 0 | 0 |
| 0 | 0 | 1 | 0 |
| 0 | 1 | 0 | 0 |
| 0 | 1 | 1 | 0 |
| 1 | 0 | 0 | 0 |
| 1 | 0 | 1 | 0 |
| 1 | 1 | 0 | 0 |
| 1 | 1 | 1 | 1 |

**v. 2-Input NAND gate:**

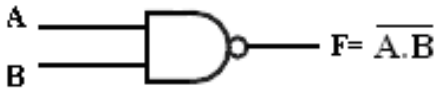



Figure 1.8 Symbol of 2-Input NAND gate

Table 1.5 Truth Table of 2-Input NAND gate

| A | B | F= $\overline{A.B}$ |
|---|---|---|
| 0 | 0 | 1 |
| 0 | 1 | 1 |
| 1 | 0 | 1 |
| 1 | 1 | 0 |

**vi. 3-Input NAND gate:**

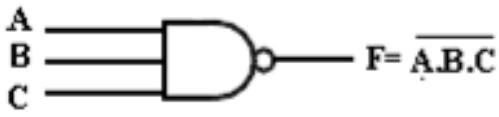



Figure 1.10 Symbol of 3-Input NAND gate

Table 1.6 Truth Table of 3-Input NAND gate

| A | B | C | F= $\overline{A.B.C}$ |
|---|---|---|---|
| 0 | 0 | 0 | 1 |
| 0 | 0 | 1 | 1 |
| 0 | 1 | 0 | 1 |
| 0 | 1 | 1 | 1 |
| 1 | 0 | 0 | 1 |
| 1 | 0 | 1 | 1 |
| 1 | 1 | 0 | 1 |
| 1 | 1 | 1 | 0 |

**vii. NOR gate:**

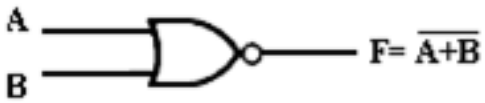



Figure 1.12 Symbol of NOR gate

Table 1.7 Truth Table of NOR gate

| A | B | F= $\overline{A+B}$ |
|---|---|---|
| 0 | 0 | 1 |
| 0 | 1 | 0 |
| 1 | 0 | 0 |
| 1 | 1 | 0 |

**viii. EX-OR gate:**

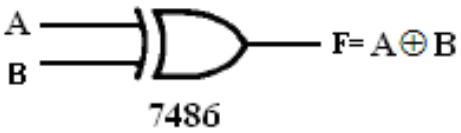



Figure 1.14 Symbol of Ex-OR gate

Table 1.8 Truth Table of Ex-OR gate

| A | B | F= A⊕B |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 1 |
| 1 | 0 | 1 |
| 1 | 1 | 0 |

**RESULT:**

Thus the logic gates are studied and their truth tables are verified.

**Ex. No. :2 : Experimental verification of Boolean function**
**Date :**

**AIM:**

To implement the given Boolean Function using Logic Gates.

$$F1 = x + y'z$$

**APPARATUS REQUIRED:**

| Sl.No | Components | Specification | Quantity |
|---|---|---|---|
| 1. | NI- Multisim | Version 14.1 | - |
| 2. | PCS | Windows 10 | 1 |

**THEORY**

It is desirable to connect gates together in as few a number as possible to create desired output result given a fixed set of input conditions. Alternatively, it may be necessary to utilize one type of gate to produce several other logic functions. Purchasing one IC type in bulk quantity has the advantage of reducing the cost of these chips. Two theories are used to facilitate these objectives.

i) Boolean Algebra and

ii) Demorgan's Theorem

Boolean algebra utilizes rules based on logic gate operations.

**Karnaugh's Map Reduction Technique**

The implication of logic circuits is based on Karnaugh's map. There are two fundamental approaches in logic design.

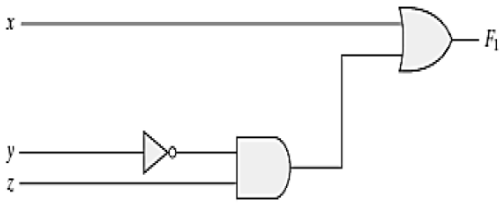
i. Product of Sum method

ii. Sum of product method

The Sum of product solution results in NAND circuit, while the Product of Sum solution results in NOR circuit corresponding to a given truth table.

The complexity of the digital logic gates can be implemented in a Boolean function. The map method provides a simple straightforward procedure for minimizing the Boolean functions. This map method is known as "Karnaugh's map".

**LOGIC DIAGRAM:**




**OBSERVATION [L=logic 0, H=logic 1]]**

| X | Y | Z | F1 |
|---|---|---|---|
| 0 | 0 | 0 | |
| 0 | 0 | 1 | |
| 0 | 1 | 0 | |
| 0 | 1 | 1 | |
| 1 | 0 | 0 | |
| 1 | 0 | 1 | |
| 1 | 1 | 0 | |
| 1 | 1 | 1 | |

**RESULT:**

Thus the given Boolean function are verified with the truth table.

**Ex. No. :3 : DESIGN AND IMPLEMENTATION OF HALF / FULL ADDER AND SUBTRACTOR**

**Date :**

---

**AIM:**

To design and construct a half adder, full adder, half subtractor and full subtractor circuits and verify their truth table using logic gates.

**APPARATUS REQUIRED:**

| Sl.No | Components | Specification | Quantity |
|---|---|---|---|
| 1. | NI- Multisim | Version 14.1 | - |
| 2. | PCS | Windows 10 | 1 |

**THEORY:**

***Half Adder:***

- A half-adder is a combinational circuit that can be used to add two binary bits.
- It has two inputs that represent the two bits to be added and two outputs, with one producing the SUM output and the other producing the CARRY. The Sum can be applied using EX-OR gate, carry output can be applied using an AND gate.

***Full Adder:***

- A full adder is a combinational circuit that forms the arithmetic sum of three input bits. It consists of 3 inputs and 2 outputs.
- Two of the input variables, represent the significant bits to be added. The third input represents the carry from previous lower significant position.
- The logic diagram of the full adder can also be implemented with two half-adders and one OR gate.
- The S output from the second half adder is the exclusive-OR of $C_{in}$ and the output of the first half-adder

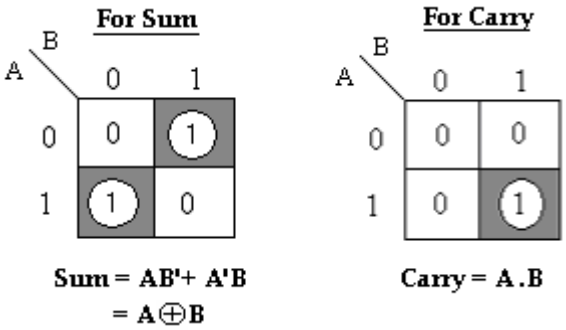
## i. HALF ADDER:

**TRUTH TABLE:**

Table 1.9 Truth Table of Half adder

| Inputs | | Outputs | |
|---|---|---|---|
| A | B | Carry (C) | Sum (S) |
| 0 | 0 | 0 | 0 |
| 0 | 1 | 0 | 1 |
| 1 | 0 | 0 | 1 |
| 1 | 1 | 1 | 0 |

**K- MAP SIMPLIFICATION:**




**LOGIC DIAGRAM:**

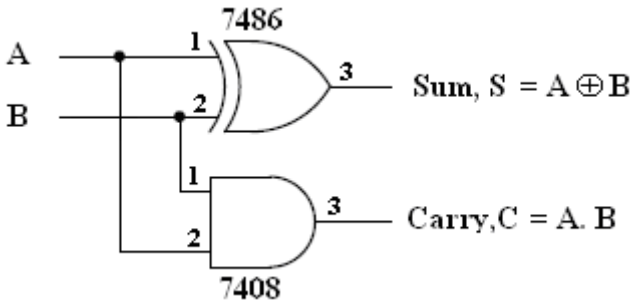



Figure 1.16 Logic Diagram of Half adder

## ii. FULL ADDER:

### TRUTH TABLE:

Table 1.10 Truth Table of Full Adder

| Inputs | | | Outputs | |
|---|---|---|---|---|
| A | B | $C_{in}$ | Sum (S) | Carry ($C_{out}$) |
| 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 1 | 1 | 0 |
| 0 | 1 | 0 | 1 | 0 |
| 0 | 1 | 1 | 0 | 1 |
| 1 | 0 | 0 | 1 | 0 |
| 1 | 0 | 1 | 0 | 1 |
| 1 | 1 | 0 | 0 | 1 |
| 1 | 1 | 1 | 1 | 1 |

**K-MAP SIMPLIFICATION:**

**For Sum**

| A \ $BC_{in}$ | 00 | 01 | 11 | 10 |
|---|---|---|---|---|
| 0 | 0 | (1) | 0 | (1) |
| 1 | (1) | 0 | (1) | 0 |

**For Carry**

| A \ $BC_{in}$ | 00 | 01 | 11 | 10 |
|---|---|---|---|---|
| 0 | 0 | 0 | 1 | 0 |
| 1 | 0 | 1 | 1 | 1 |

**Sum, S** $= A'B'C_{in} + A'BC'_{in} + AB'C'_{in} + ABC_{in}$

$= C_{in} \oplus (A \oplus B)$

**Carry,** $C_{out} = AB + AC_{in} + BC_{in}$

**LOGIC DIAGRAM:**

**Full Adder**:

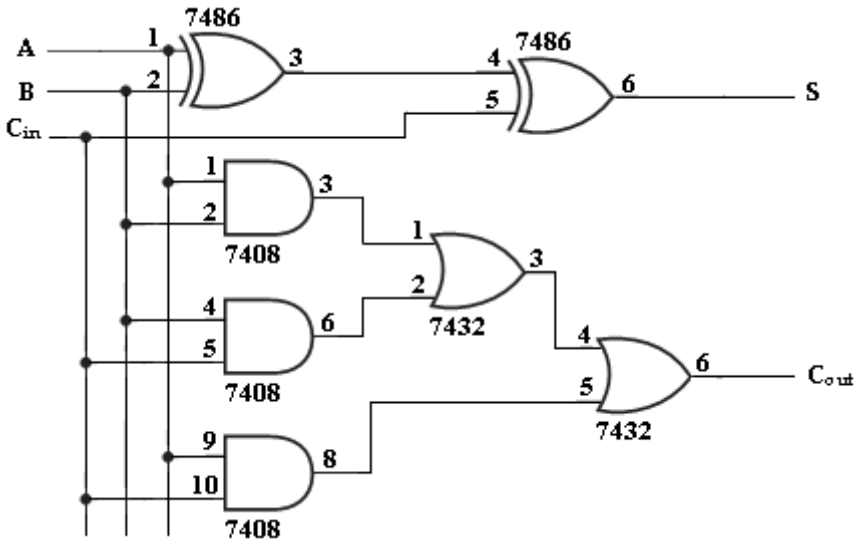



Figure 1.17 Logic Diagram of Full adder

***Half Subtractor:***

- A half-subtractor is a combinational circuit that can be used to subtract one binary digit from another to produce a DIFFERENCE output and a BORROW output.
- The BORROW output here specifies whether a '1' has been borrowed to perform the subtraction.
- The difference can be applied using EX-OR gate, borrow output can be applied using an AND gate and an inverter.

***Full Subtractor:***

- A full subtractor performs subtraction operation on two bits, a minuend and a subtrahend, and also takes into consideration whether a '1' has already been borrowed by the previous adjacent lower minuend bit or not.
- As a result, there are three bits to be handled at the input of a full subtractor, namely the two bits to be subtracted and a borrow bit designated as $B_{in}$.
- There are two outputs, namely the DIFFERENCE output D and the BORROW output $B_o$.
- The BORROW output bit tells whether the minuend bit needs to borrow a '1' from the next possible higher minuend bit.

**Full Adder using Two Half Adders:**

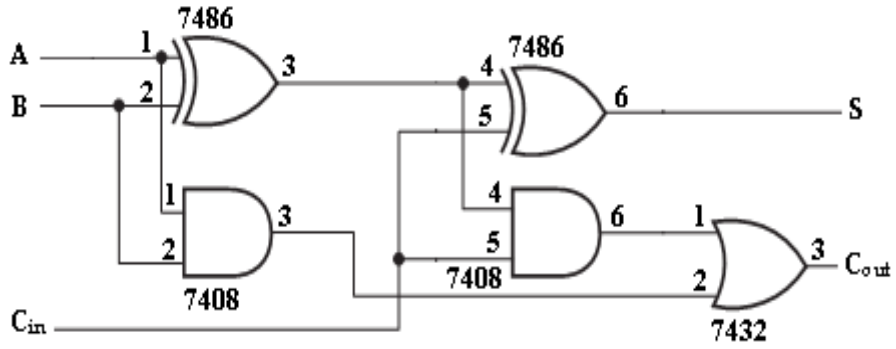



Figure 1.18 Logic Diagram of Full Adder using Two Half Adders

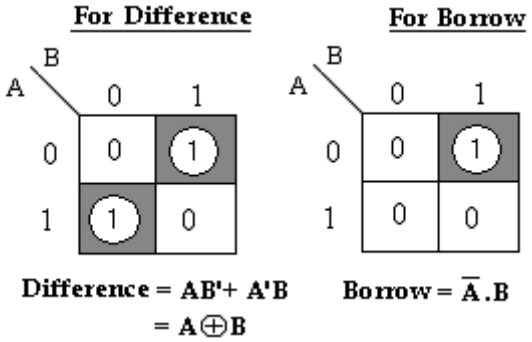
## iii. HALF SUBTRACTOR:

**TRUTH TABLE:**

Table 1.11 Truth Table of Half Subtractor

| Input | | Output | |
|---|---|---|---|
| A | B | Difference (D) | Borrow ($B_{out}$) |
| 0 | 0 | 0 | 0 |
| 0 | 1 | 1 | 1 |
| 1 | 0 | 1 | 0 |
| 1 | 1 | 0 | 0 |

**K- MAP SIMPLIFICATION:**




**LOGIC DIAGRAM:**

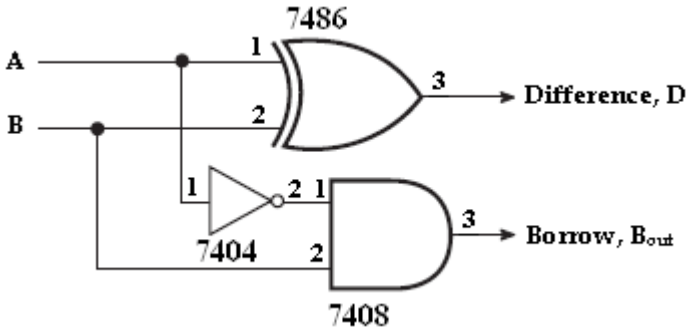



Figure 1.19 Logic Diagram of Half subtractor

## iv. FULL SUBTRACTOR:

**TRUTH TABLE:**

Table 1.12 Truth Table of Full Adder

| Inputs | | | Outputs | |
|---|---|---|---|---|
| A | B | $B_{in}$ | Difference(D) | Borrow($B_{out}$) |
| 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 1 | 1 | 1 |
| 0 | 1 | 0 | 1 | 1 |
| 0 | 1 | 1 | 0 | 1 |
| 1 | 0 | 0 | 1 | 0 |
| 1 | 0 | 1 | 0 | 0 |
| 1 | 1 | 0 | 0 | 0 |
| 1 | 1 | 1 | 1 | 1 |

**K- MAP SIMPLIFICATION:**

**For Difference**

| A \ $BB_{in}$ | 00 | 01 | 11 | 10 |
|---|---|---|---|---|
| 0 | 0 | (1) | 0 | (1) |
| 1 | (1) | 0 | (1) | 0 |

**For Borrow**

| A \ $BB_{in}$ | 00 | 01 | 11 | 10 |
|---|---|---|---|---|
| 0 | 0 | (1) | (1) | 1 |
| 1 | 0 | 0 | (1) | 0 |

**Difference, $D = A'B'B_{in} + A'BB'_{in} + AB'B'_{in} + ABB_{in}$**

**$= B_{in} \oplus (A \oplus B)$**

**Borrow, $B_{out} = A'B + A'B_{in} + BB_{in}$**

**LOGIC DIAGRAM:**

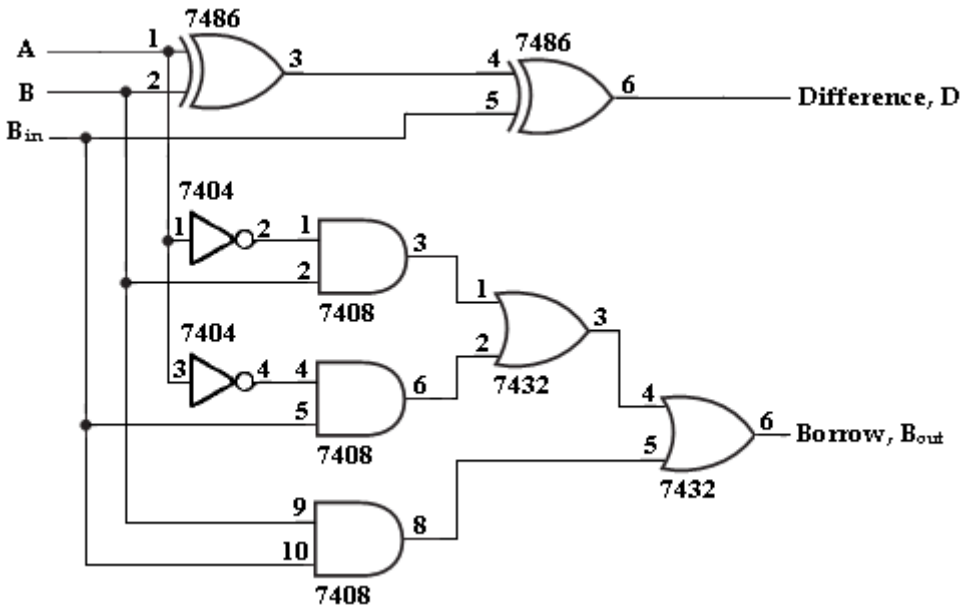



Figure 1.20 Logic Diagram of Full subtractor

**Full Subtractor with Two Half Subtractors:**

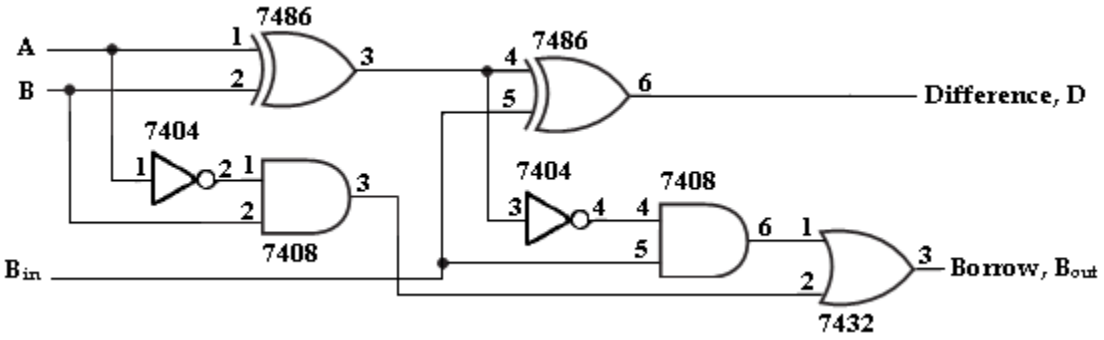



Figure 1.21 Logic Diagram of Full subtractor with Two Half Subtractors

**PROCEDURE:**

1. Connections are given as per the logic diagram.
2. Logic inputs are given as per the truth table.
3. Observe the logic output and verify with their truth tables**.**

**RESULT:**

Thus half adder, full adder, half subtractor and full subtractor circuits was designed using logic gates and their truth tables were verified.

**Ex. No. :4 : DESIGN AND IMPLEMENTATION OF COMBINATIONAL CIRCUITS USING BASIC GATES FOR CODE CONVERTERS**

**Date :**

**AIM:**

To design and implement 4-bit

1. Binary to Gray code Converter
2. Gray to Binary code Converter
3. BCD to Excess-3 code Converter
4. Excess-3 code to BCD Converter

**APPARATUS REQUIRED:**

| Sl.No | Components | Specification | Quantity |
|---|---|---|---|
| 1. | NI- Multisim | Version 14.1 | - |
| 2. | PCS | Windows 10 | 1 |

**THEORY:**

- An availability of large variety of codes for the same discrete elements of information results in the use of different codes by different systems.
- A conversion circuit must be inserted between the two systems if each uses different codes for the same information. Thus, code converter is a circuit that makes the two systems compatible even though each uses different binary code.
- The input variable are designed as $B_3$,$B_2$,$B_1$,$B_0$ and the output variables are designed as $G_3$,$G_2$,$G_1$,$G_0$. From the truth table, combinational circuit is designed. The Boolean functions are obtained from K-Map for each output variable.
- To convert from binary code to Excess-3 code, the input lines must supply the bit combination of elements as specified by code and the output lines generate the corresponding bit combination of code.
- Each one of the four maps represents one of the four outputs of the circuit as a function of the four input variables.
- A two-level logic diagram may be obtained directly from the Boolean expressions derived by the maps. These are various other possibilities for a logic diagram that implements this circuit.

## BINARY TO GRAY CODE CONVERTER:

### TRUTH TABLE:

Table 1.13 Truth Table of Binary to Gray Code converter

| Binary code | | | | Gray code | | | |
|---|---|---|---|---|---|---|---|
| $B_3$ | $B_2$ | $B_1$ | $B_0$ | $G_3$ | $G_2$ | $G_1$ | $G_0$ |
| 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 0 | 1 | 0 | 0 | 0 | 1 |
| 0 | 0 | 1 | 0 | 0 | 0 | 1 | 1 |
| 0 | 0 | 1 | 1 | 0 | 0 | 1 | 0 |
| 0 | 1 | 0 | 0 | 0 | 1 | 1 | 0 |
| 0 | 1 | 0 | 1 | 0 | 1 | 1 | 1 |
| 0 | 1 | 1 | 0 | 0 | 1 | 0 | 1 |
| 0 | 1 | 1 | 1 | 0 | 1 | 0 | 0 |
| 1 | 0 | 0 | 0 | 1 | 1 | 0 | 0 |
| 1 | 0 | 0 | 1 | 1 | 1 | 0 | 1 |
| 1 | 0 | 1 | 0 | 1 | 1 | 1 | 1 |
| 1 | 0 | 1 | 1 | 1 | 1 | 1 | 0 |
| 1 | 1 | 0 | 0 | 1 | 0 | 1 | 0 |
| 1 | 1 | 0 | 1 | 1 | 0 | 1 | 1 |
| 1 | 1 | 1 | 0 | 1 | 0 | 0 | 1 |
| 1 | 1 | 1 | 1 | 1 | 0 | 0 | 0 |

### K- MAP SIMPLIFICATION:

For $G_3$

| $B_3B_2$ \ $B_1B_0$ | 00 | 01 | 11 | 10 |
|---|---|---|---|---|
| 00 | 0 | 0 | 0 | 0 |
| 01 | 0 | 0 | 0 | 0 |
| 11 | 1 | 1 | 1 | 1 |
| 10 | 1 | 1 | 1 | 1 |

$G_3 = B_3$

For $G_2$

| $B_3B_2$ \ $B_1B_0$ | 00 | 01 | 11 | 10 |
|---|---|---|---|---|
| 00 | 0 | 0 | 0 | 0 |
| 01 | 1 | 1 | 1 | 1 |
| 11 | 0 | 0 | 0 | 0 |
| 10 | 1 | 1 | 1 | 1 |

$G_2 = B_3'B_2 + B_3B_2'$
$= B_3 \oplus B_2$

For $G_1$

| $B_3B_2$ \ $B_1B_0$ | 00 | 01 | 11 | 10 |
|---|---|---|---|---|
| 00 | 0 | 0 | 1 | 1 |
| 01 | 1 | 1 | 0 | 0 |
| 11 | 1 | 1 | 0 | 0 |
| 10 | 0 | 0 | 1 | 1 |

$G_1 = B_2'B_1 + B_2B_1'$
$= B_2 \oplus B_1$

For $G_0$

| $B_3B_2$ \ $B_1B_0$ | 00 | 01 | 11 | 10 |
|---|---|---|---|---|
| 00 | 0 | 1 | 0 | 1 |
| 01 | 0 | 1 | 0 | 1 |
| 11 | 0 | 1 | 0 | 1 |
| 10 | 0 | 1 | 0 | 1 |

$G_0 = B_1'B_0 + B_1B_0'$
$= B_1 \oplus B_0$

**LOGIC DIAGRAM:**

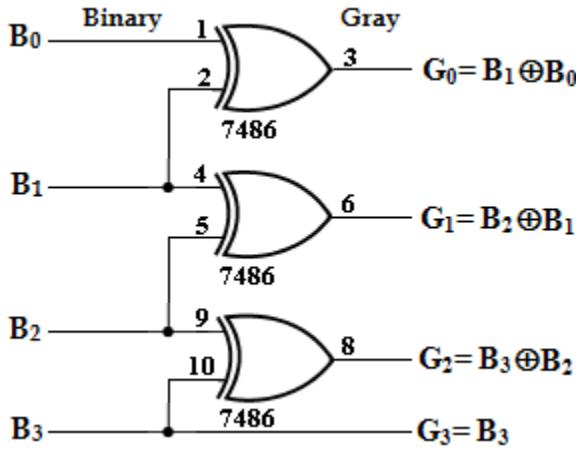



Figure 1.22 Logic Diagram of Binary to Gray Code converter

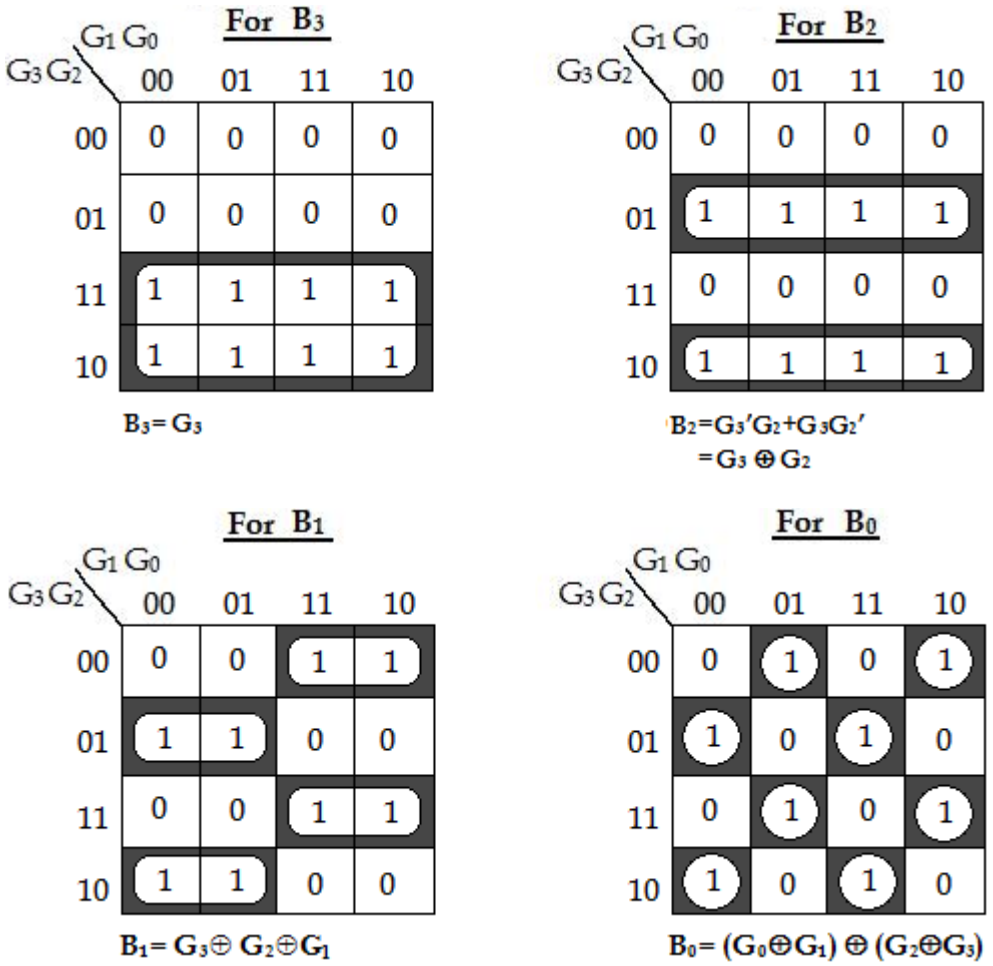
## GRAY TO BINARY CODE CONVERTER:

**TRUTH TABLE:**

Table 1.14 Truth Table of Gray to Binary Code converter

| Gray code | | | | Binary code | | | |
|---|---|---|---|---|---|---|---|
| $G_3$ | $G_2$ | $G_1$ | $G_0$ | $B_3$ | $B_2$ | $B_1$ | $B_0$ |
| 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 0 | 1 | 0 | 0 | 0 | 1 |
| 0 | 0 | 1 | 0 | 0 | 0 | 1 | 1 |
| 0 | 0 | 1 | 1 | 0 | 0 | 1 | 0 |
| 0 | 1 | 0 | 0 | 0 | 1 | 1 | 1 |
| 0 | 1 | 0 | 1 | 0 | 1 | 1 | 0 |
| 0 | 1 | 1 | 0 | 0 | 1 | 0 | 0 |
| 0 | 1 | 1 | 1 | 0 | 1 | 0 | 1 |
| 1 | 0 | 0 | 0 | 1 | 1 | 1 | 1 |
| 1 | 0 | 0 | 1 | 1 | 1 | 1 | 0 |
| 1 | 0 | 1 | 0 | 1 | 1 | 0 | 0 |
| 1 | 0 | 1 | 1 | 1 | 1 | 0 | 1 |
| 1 | 1 | 0 | 0 | 1 | 0 | 0 | 0 |
| 1 | 1 | 0 | 1 | 1 | 0 | 0 | 1 |
| 1 | 1 | 1 | 0 | 1 | 0 | 1 | 1 |
| 1 | 1 | 1 | 1 | 1 | 0 | 1 | 0 |

**K-MAP SIMPLIFICATION:**




**LOGIC DIAGRAM:**

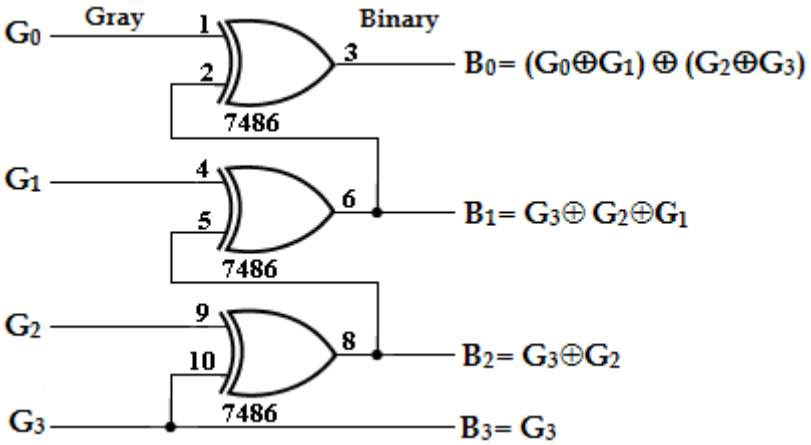



Figure 1.23 Logic Diagram of Gray to Binary Code converter

## BCD TO EXCESS-3 CODE:

### TRUTH TABLE:

Table 1.15 Truth Table of BCD to Excess-3 converter

| BCD code | | | | Excess-3 code | | | |
|---|---|---|---|---|---|---|---|
| $B_3$ | $B_2$ | $B_1$ | $B_0$ | $E_3$ | $E_2$ | $E_1$ | $E_0$ |
| 0 | 0 | 0 | 0 | 0 | 0 | 1 | 1 |
| 0 | 0 | 0 | 1 | 0 | 1 | 0 | 0 |
| 0 | 0 | 1 | 0 | 0 | 1 | 0 | 1 |
| 0 | 0 | 1 | 1 | 0 | 1 | 1 | 0 |
| 0 | 1 | 0 | 0 | 0 | 1 | 1 | 1 |
| 0 | 1 | 0 | 1 | 1 | 0 | 0 | 0 |
| 0 | 1 | 1 | 0 | 1 | 0 | 0 | 1 |
| 0 | 1 | 1 | 1 | 1 | 0 | 1 | 0 |
| 1 | 0 | 0 | 0 | 1 | 0 | 1 | 1 |
| 1 | 0 | 0 | 1 | 1 | 1 | 0 | 0 |

### K-MAP SIMPLIFICATION:

For $E_3$

| $B_3B_2$ \ $B_1B_0$ | 00 | 01 | 11 | 10 |
|---|---|---|---|---|
| 00 | 0 | 0 | 0 | 0 |
| 01 | 0 | 1 | 1 | 1 |
| 11 | x | x | x | x |
| 10 | 1 | 1 | x | x |

$E_3 = B_3 + B_2 (B_0 + B_1)$

For $E_2$

| $B_3B_2$ \ $B_1B_0$ | 00 | 01 | 11 | 10 |
|---|---|---|---|---|
| 00 | 0 | 1 | 1 | 1 |
| 01 | 1 | 0 | 0 | 0 |
| 11 | x | x | x | x |
| 10 | 0 | 1 | x | x |

$E_2 = B_2B_1'B_0' + B_2' (B_0 + B_1)$

For $E_1$

| $B_3B_2$ \ $B_1B_0$ | 00 | 01 | 11 | 10 |
|---|---|---|---|---|
| 00 | 1 | 0 | 1 | 0 |
| 01 | 1 | 0 | 1 | 0 |
| 11 | x | x | x | x |
| 10 | 1 | 0 | x | x |

$E_1 = B_1'B_0' + B_1B_0$

$= B_1 \odot B_0$

For $E_0$

| $B_3B_2$ \ $B_1B_0$ | 00 | 01 | 11 | 10 |
|---|---|---|---|---|
| 00 | 1 | 0 | 0 | 1 |
| 01 | 1 | 0 | 0 | 1 |
| 11 | x | x | x | x |
| 10 | 1 | 0 | x | x |

$E_0 = B_0'$

**LOGIC DIAGRAM:**

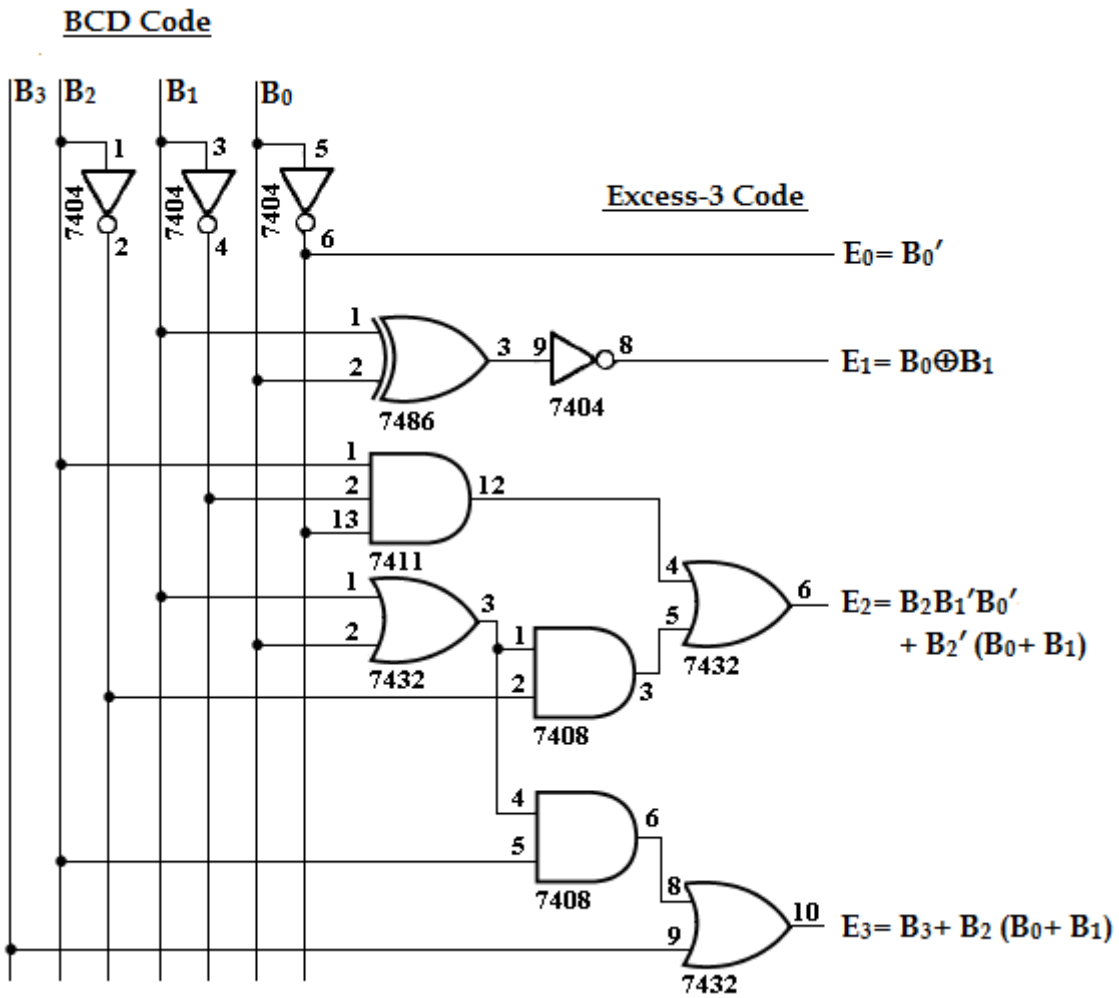



Figure 1.24 Logic Diagram of BCD to Excess-3 converter

**EXCESS-3 TO BCD CONVERTER:**

**TRUTH TABLE:**

Table 1.16 Truth Table of Excess-3 to BCD converter

| Excess-3 code | | | | BCD code | | | |
|---|---|---|---|---|---|---|---|
| $E_3$ | $E_2$ | $E_1$ | $E_0$ | $B_3$ | $B_2$ | $B_1$ | $B_0$ |
| 0 | 0 | 1 | 1 | 0 | 0 | 0 | 0 |
| 0 | 1 | 0 | 0 | 0 | 0 | 0 | 1 |
| 0 | 1 | 0 | 1 | 0 | 0 | 1 | 0 |
| 0 | 1 | 1 | 0 | 0 | 0 | 1 | 1 |
| 0 | 1 | 1 | 1 | 0 | 1 | 0 | 0 |
| 1 | 0 | 0 | 0 | 0 | 1 | 0 | 1 |
| 1 | 0 | 0 | 1 | 0 | 1 | 1 | 0 |
| 1 | 0 | 1 | 0 | 0 | 1 | 1 | 1 |
| 1 | 0 | 1 | 1 | 1 | 0 | 0 | 0 |
| 1 | 1 | 0 | 0 | 1 | 0 | 0 | 1 |

**K-MAP SIMPLIFICATION:**

For $B_3$

| $E_3E_2$ \ $E_1E_0$ | 00 | 01 | 11 | 10 |
|---|---|---|---|---|
| 00 | X | X | 0 | X |
| 01 | 0 | 0 | 0 | 0 |
| 11 | 1 | X | X | X |
| 10 | 0 | 0 | 1 | 0 |

$B_3 = E_3E_2 + E_3E_1E_0$

For $B_2$

| $E_3E_2$ \ $E_1E_0$ | 00 | 01 | 11 | 10 |
|---|---|---|---|---|
| 00 | X | X | 0 | X |
| 01 | 0 | 0 | 1 | 0 |
| 11 | 0 | X | X | X |
| 10 | 1 | 1 | 0 | 1 |

$B_2 = E_2'E_1' + E_2E_1E_0 + E_2'E_0'$

For $B_1$

| $E_3E_2$ \ $E_1E_0$ | 00 | 01 | 11 | 10 |
|---|---|---|---|---|
| 00 | X | X | 0 | X |
| 01 | 0 | 1 | 0 | 1 |
| 11 | 0 | X | X | X |
| 10 | 0 | 1 | 0 | 1 |

$B_1 = E_1'E_0 + E_1E_0'$
$= E_1 \oplus E_0$

For $B_0$

| $E_3E_2$ \ $E_1E_0$ | 00 | 01 | 11 | 10 |
|---|---|---|---|---|
| 00 | X | X | 0 | X |
| 01 | 1 | 0 | 0 | 1 |
| 11 | 1 | X | X | X |
| 10 | 1 | 0 | 0 | 1 |

$B_0 = E_0'$

**LOGIC DIAGRAM:**

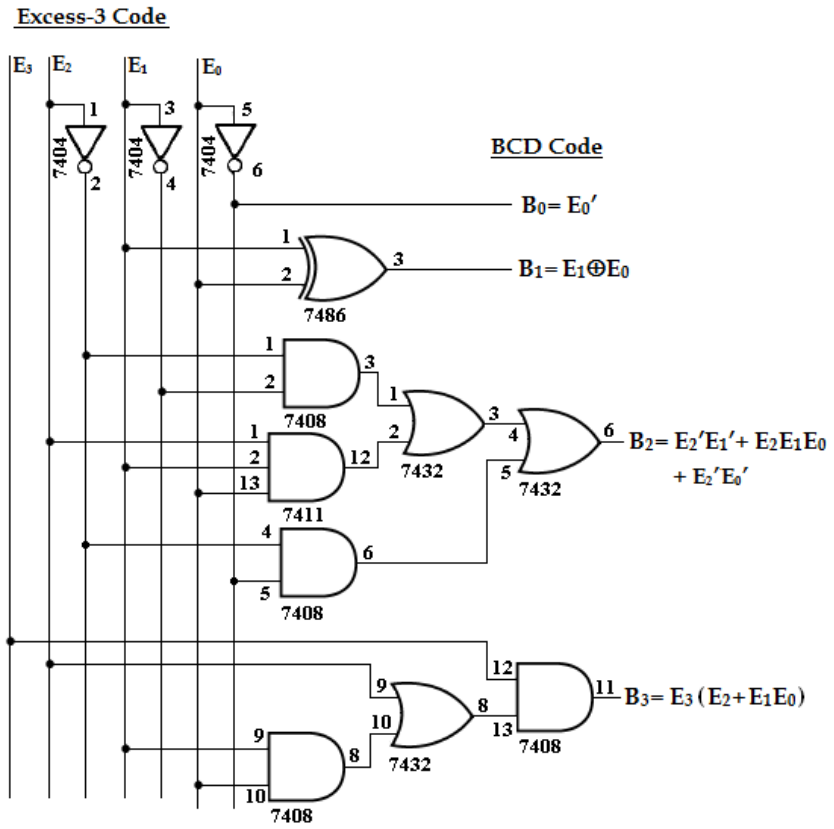



Figure 1.25 Logic Diagram of Excess-3 to BCD converter

**PROCEDURE**:

1. Connections are given as per the logic diagram.
2. Logic inputs are given as per the truth table.
3. Observe the logic output and verify with the truth tables.

**RESULT:**

Thus the 4-bit Binary to Gray code Converter, Gray to Binary code Converter, BCD to Excess-3 code Converter, Excess-3 code to BCD Converter was designed and implemented.

**Ex. No. :5 : DESIGN AND IMPLEMENTATION OF MULTIPLEXER AND DEMULTIPLEXER**

**Date :**

**AIM:**

To design and implement multiplexer and demultiplexer using logic gates.

**APPARATUS REQUIRED:**

| Sl.No | Components | Specification | Quantity |
|---|---|---|---|
| 1. | NI- Multisim | Version 14.1 | - |
| 2. | PCS | Windows 10 | 1 |

**THEORY:**

***Multiplexer:***

- Multiplexer means transmitting a large number of information units over a small number of channels or lines.
- A digital multiplexer is a combinational circuit that selects binary information from one of many input lines and directs it to a single output line.
- The selection of a particular input line is controlled by a set of selection lines.
- Normally there are $2^n$ input line and 'n' selection lines whose bit combination determine which input is selected. It is called as **data selector**, because the output depends on the input data bit that is selected.

***Demultiplexer:***

- The function of Demultiplexer is in contrast to multiplexer function. It takes information from one line and distributes it to a given number of output lines.
- For this reason, the demultiplexer is also known as a **data distributor**. Decoder can also be used as Demultiplexer.
- In the 1:4 demultiplexer circuit, the data input line goes to all of the AND gates.
- The data select lines enable only one gate at a time and the data on the data input line will pass through the selected gate to the associated data output line.

## i. 4:1 MULTIPLEXER:

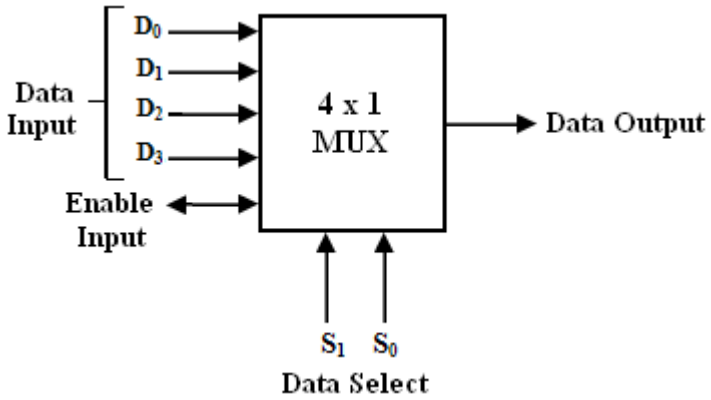



Figure 2.1 Pin Diagram of Multiplexer

**FUNCTION TABLE:**

Table 2.1 Function Table of Multiplexer

| $S_1$ | $S_0$ | INPUTS, (Y) |
|---|---|---|
| 0 | 0 | $D_0 = D_0 S_1'S_0'$ |
| 0 | 1 | $D_1 = D_1 S_1'S_0$ |
| 1 | 0 | $D_2 = D_2 S_1S_0'$ |
| 1 | 1 | $D_3 = D_3 S_1S_0$ |

**Output, $Y = D_0 S_1'S_0' + D_1 S_1'S_0 + D_2 S_1S_0' + D_3 S_1S_0$**

**LOGIC DIAGRAM:**

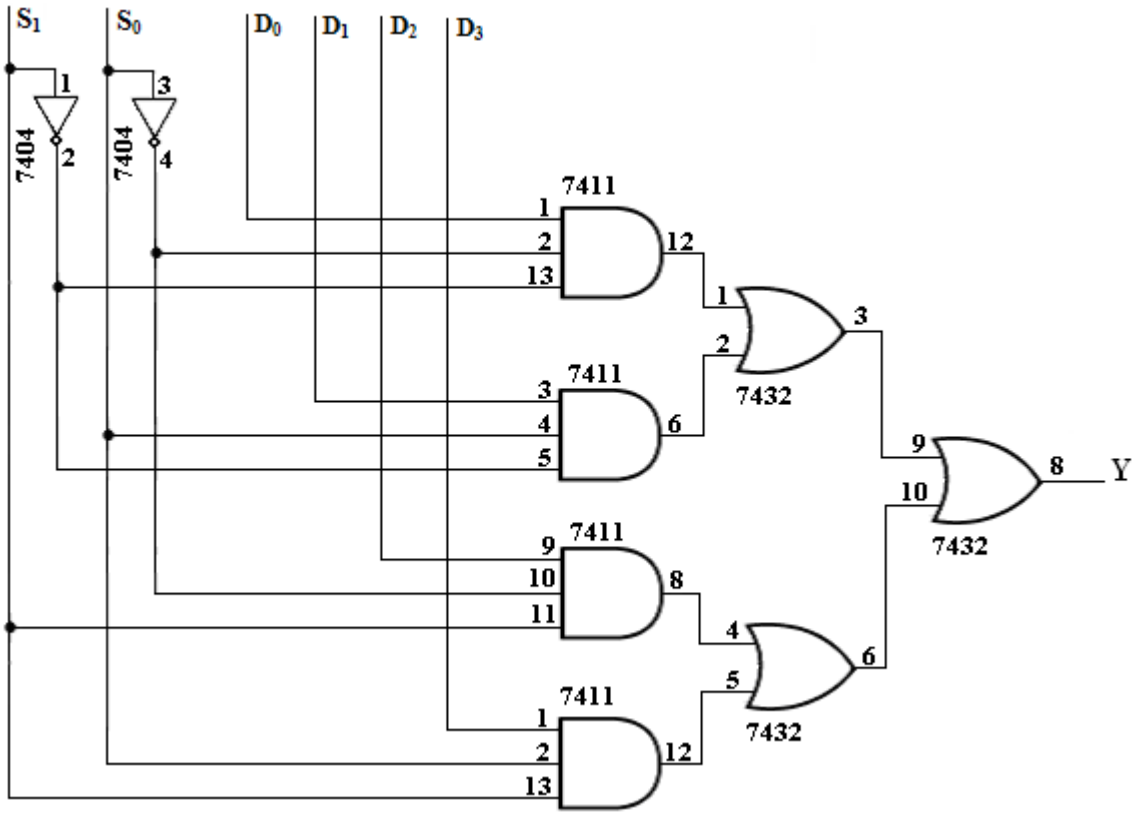



Figure 2.2 Logic Diagram of Multiplexer

**TRUTH TABLE:**

Table 2.2 Truth Table of Multiplexer

| $S_1$ | $S_0$ | OUTPUTS, (Y) |
|---|---|---|
| 0 | 0 | $D_0$ |
| 0 | 1 | $D_1$ |
| 1 | 0 | $D_2$ |
| 1 | 1 | $D_3$ |

## ii. 1: 4 DEMULTIPLEXER:

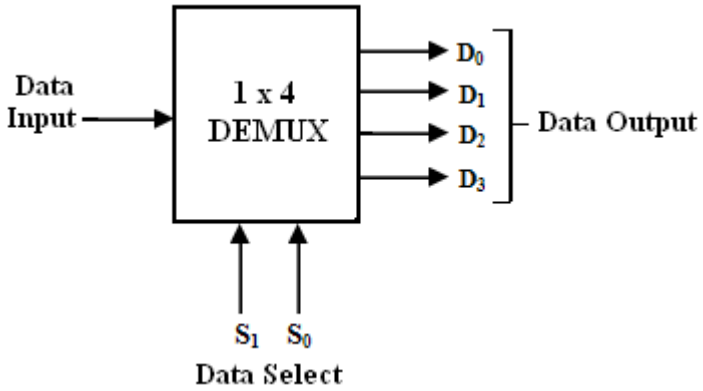



Figure 2.3 Pin Diagram of Demultiplexer

**FUNCTION TABLE:**

Table 2.3 Function Table of Demultiplexer

| $S_1$ | $S_0$ | INPUT, X |
|---|---|---|
| 0 | 0 | $D_0= XS_1'S_0'$ |
| 0 | 1 | $D_1= XS_1'S_0$ |
| 1 | 0 | $D_2= XS_1S_0'$ |
| 1 | 1 | $D_3= XS_1S_0$ |

**Output, $Y= XS_1'S_0' + XS_1'S_0 + XS_1S_0' + XS_1S_0$**

**LOGIC DIAGRAM:**

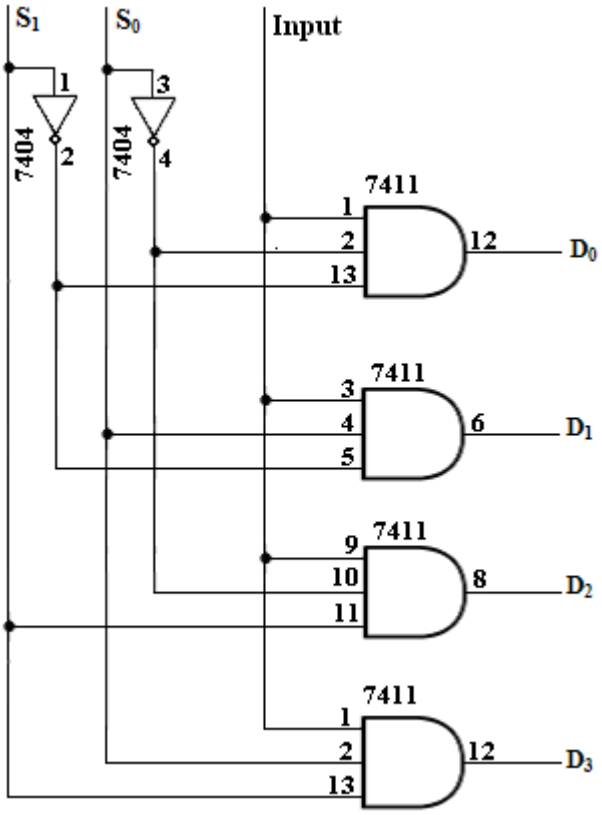



Figure 2.4 Logic Diagram of Demultiplexer

**TRUTH TABLE:**

Table 2.4 Truth Table of Demultiplexer

| INPUT | | | OUTPUT | | | |
|---|---|---|---|---|---|---|
| $S_1$ | $S_0$ | I/P | $D_0$ | $D_1$ | $D_2$ | $D_3$ |
| 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 1 | 1 | 0 | 0 | 0 |
| 0 | 1 | 0 | 0 | 0 | 0 | 0 |
| 0 | 1 | 1 | 0 | 1 | 0 | 0 |
| 1 | 0 | 0 | 0 | 0 | 0 | 0 |
| 1 | 0 | 1 | 0 | 0 | 1 | 0 |
| 1 | 1 | 0 | 0 | 0 | 0 | 0 |
| 1 | 1 | 1 | 0 | 0 | 0 | 1 |

**PROCEDURE:**

1. Connections are given as per the logic diagram.
2. Logic inputs are given as per the truth table.
3. Observe the logic output and verify with the truth tables.

**RESULT:**

Thus the multiplexer and demultiplexer was designed and implemented using logic gates was verified.

**Ex. No. :6 : DESIGN AND IMPLEMENTATION OF ENCODER AND DECODER**

**Date :**

**AIM:**

To design and implementation encoder and decoder using logic gates.

**APPARATUS REQUIRED:**

| Sl.No | Components | Specification | Quantity |
|---|---|---|---|
| 1. | NI- Multisim | Version 14.1 | - |
| 2. | PCS | Windows 10 | 1 |

**THEORY:**

***Encoder:***

- An encoder is a digital circuit that performs inverse operation of a decoder. An encoder has $2^n$ input lines and 'n' output lines.
- In encoder the output lines generates the binary code corresponding to the input value.
- In octal to binary encoder it has eight inputs, one for each octal digit and three output that generates the corresponding binary code.
- In encoder it is assumed that only one input has a value of one at any given time otherwise the circuit is meaningless. It has an ambiguila that when all inputs are zero the outputs are zero. The zero outputs can also be generated when D0=1.

***Decoder:***

- A decoder is a multiple output logic circuit which converts input into coded output where input and output codes are different.
- The input code generally has few bits than the output code. Each input code word produces a different output code word i.e., there is one to one mapping can be expressed in truth table.
- In block diagram of decoder circuit the encoded information is present as n input producing $2^n$ possible outputs. The $2^n$ output values are from 0 through out $2^n$-1.

## i. DECODER:

**Logic Diagram ( 2-to-4- Line Decoder with Enable Input):**

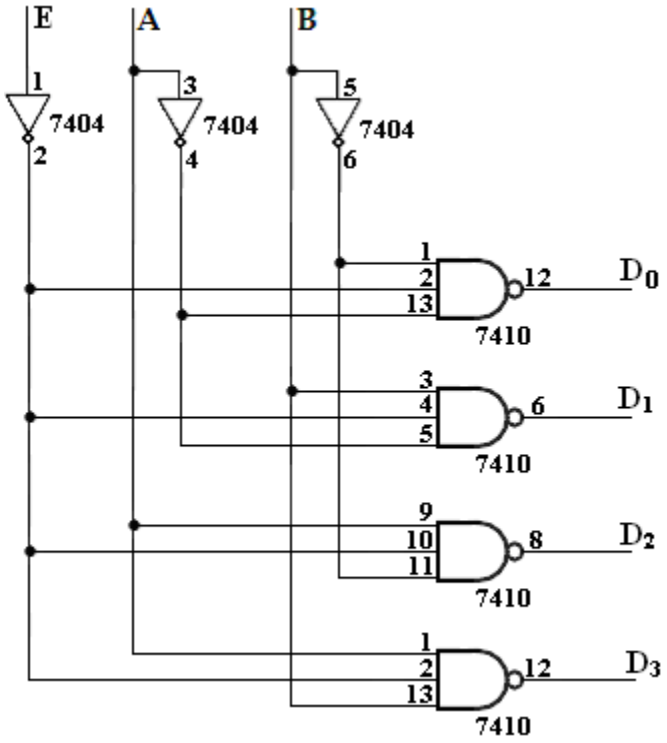



Figure 3.1 Logic Diagram of 2-to-4- Line Decoder with Enable Input

**Truth Table:**

Table 3.1 Truth Table of Decoder

| INPUTS | | | OUTPUTS | | | |
|---|---|---|---|---|---|---|
| **E** | **A** | **B** | **$D_3$** | **$D_1$** | **$D_2$** | **$D_3$** |
| 1 | X | X | 1 | 1 | 1 | 1 |
| 0 | 0 | 0 | 0 | 1 | 1 | 1 |
| 0 | 0 | 1 | 1 | 0 | 1 | 1 |
| 0 | 1 | 0 | 1 | 1 | 0 | 1 |
| 0 | 1 | 1 | 1 | 1 | 1 | 0 |

## ii. ENCODER:

**Logic Diagram:**

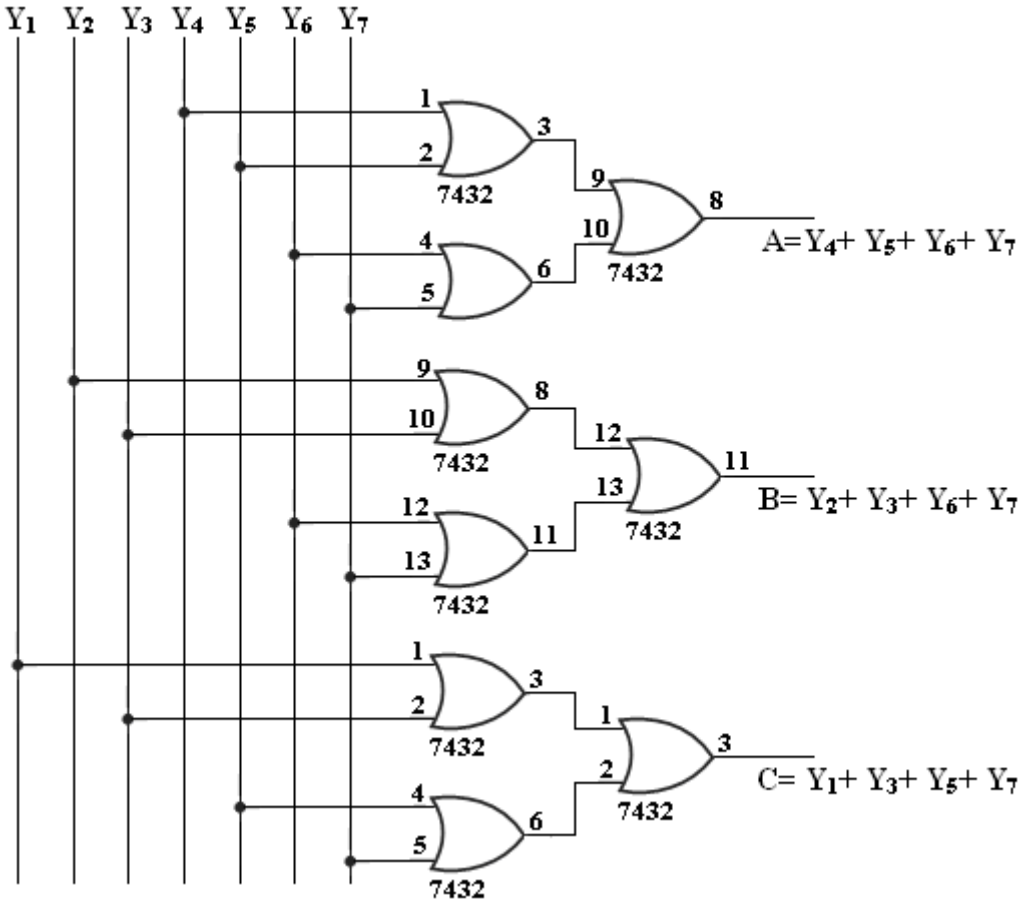



Figure 3.2 Logic Diagram of 8:3 encoder

**Truth Table:**

Table 3.2 Truth Table of 8:3 encoder

| INPUTS | | | | | | | OUTPUTS | | |
|---|---|---|---|---|---|---|---|---|---|
| $Y_1$ | $Y_2$ | $Y_3$ | $Y_4$ | $Y_5$ | $Y_6$ | $Y_7$ | A | B | C |
| 1 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 1 |
| 0 | 1 | 0 | 0 | 0 | 0 | 0 | 0 | 1 | 0 |
| 0 | 0 | 1 | 0 | 0 | 0 | 0 | 0 | 1 | 1 |
| 0 | 0 | 0 | 1 | 0 | 0 | 0 | 1 | 0 | 0 |
| 0 | 0 | 0 | 0 | 1 | 0 | 0 | 1 | 0 | 1 |
| 0 | 0 | 0 | 0 | 0 | 1 | 0 | 1 | 1 | 0 |
| 0 | 0 | 0 | 0 | 0 | 0 | 1 | 1 | 1 | 1 |

**PROCEDURE:**

1. Connections are given as per the logic diagram.
2. Logic inputs are given as per the truth table.
3. Observe the logic output and verify with the truth tables**.**

**RESULT:**

Thus the design and implementation of encoder and decoder using logic gates was done successfully.

**Ex. No. 6 : DESIGN AND IMPLEMENTATION OF 3-BIT SYNCHRONOUS UP/DOWN COUNTER**

**Date :**

**AIM:**

To design and implement 3 bit synchronous up/down counter using JK flip-flop.

**APPARATUS REQUIRED:**

| Sl.No | Components | Specification | Quantity |
|---|---|---|---|
| 1. | NI- Multisim | Version 14.1 | - |
| 2. | PCS | Windows 10 | 1 |

**THEORY:**

- A Counter is a register capable of counting number of clock pulse arriving at its clock input. Counter represents the number of clock pulses arrived.
- An up/down counter is one that is capable of progressing in increasing order or decreasing order through a certain sequence. An up/down counter is also called bi-directional counter.
- Usually up/down operation of the counter is controlled by up/down signal.
- When this signal high counter goes through up sequence and when up/down signal is low counter follows reverse sequence.
- The counter counts upwards when UP control are logic '1' and DOWN control is logic '0'. In this case the clock input of each flip-flop other than the LSB flip-flop is fed from the normal output of the immediately preceding flip-flop.
- The counter counts downwards when the UP controls input are logic '0' and DOWN control is logic '1'. In this case, the clock input of each flip-flop other than the LSB flip-flop is fed from the complemented output of the immediately preceding flip-flop.

**PROCEDURE:**

1. Connections are given as per the logic diagram.
2. Logic inputs are given as per the logic diagram.
3. Observe the logic output and verify with the truth tables.

\

## 3- BIT SYNCHRONOUS UP/DOWN COUNTER:

**STATE DIAGRAM:**

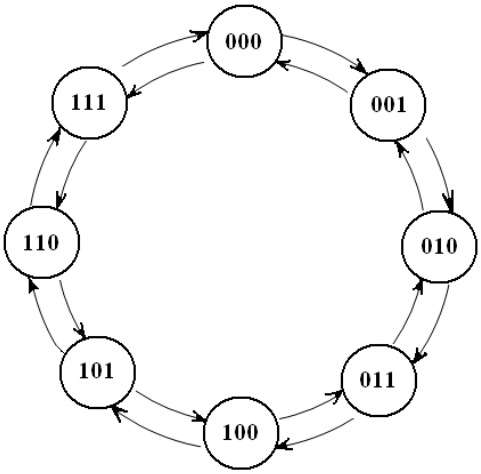



Figure 5.1 State Diagram of 3 bit synchronous up/down counter

**TRUTH TABLE:**

Table 5.1 Truth Table of 3 bit synchronous up/down counter

| Input | Present State | | | Next State | | | A | | B | | C | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Up/Down' | $Q_A$ | $Q_B$ | $Q_C$ | $Q_{A+1}$ | $Q_{B+1}$ | $Q_{C+1}$ | $J_A$ | $K_A$ | $J_B$ | $K_B$ | $J_C$ | $K_C$ |
| 0 | 0 | 0 | 0 | 1 | 1 | 1 | 1 | x | 1 | x | 1 | x |
| 0 | 1 | 1 | 1 | 1 | 1 | 0 | x | 0 | X | 0 | x | 1 |
| 0 | 1 | 1 | 0 | 1 | 0 | 1 | x | 0 | X | 1 | 1 | x |
| 0 | 1 | 0 | 1 | 1 | 0 | 0 | x | 0 | 0 | x | x | 1 |
| 0 | 1 | 0 | 0 | 0 | 1 | 1 | x | 1 | 1 | x | 1 | x |
| 0 | 0 | 1 | 1 | 0 | 1 | 0 | 0 | x | X | 0 | x | 1 |
| 0 | 0 | 1 | 0 | 0 | 0 | 1 | 0 | x | X | 1 | 1 | x |
| 0 | 0 | 0 | 1 | 0 | 0 | 0 | 0 | x | 0 | x | x | 1 |
| 1 | 0 | 0 | 0 | 0 | 0 | 1 | 0 | x | 0 | x | 1 | x |
| 1 | 0 | 0 | 1 | 0 | 1 | 0 | 0 | x | 1 | x | x | 1 |
| 1 | 0 | 1 | 0 | 0 | 1 | 1 | 0 | x | X | 0 | 1 | x |
| 1 | 0 | 1 | 1 | 1 | 0 | 0 | 1 | x | X | 1 | x | 1 |
| 1 | 1 | 0 | 0 | 1 | 0 | 1 | x | 0 | 0 | x | 1 | x |
| 1 | 1 | 0 | 1 | 1 | 1 | 0 | x | 0 | 1 | x | x | 1 |
| 1 | 1 | 1 | 0 | 1 | 1 | 1 | x | 0 | X | 0 | 1 | x |
| 1 | 1 | 1 | 1 | 0 | 0 | 0 | x | 1 | X | 1 | x | 1 |

**EXCITATION TABLE: (JK Flip-Flop)**

Table 5.2 Excitation Table of 3 bit synchronous up/down counter

| Q | $Q_{t+1}$ | J | K |
|---|---|---|---|
| 0 | 0 | 0 | x |
| 0 | 1 | 1 | x |
| 1 | 0 | x | 1 |
| 1 | 1 | x | 0 |

**K-MAP SIMPLIFICATION:**

**For $J_A$**

| UD QA \ QB QC | 00 | 01 | 11 | 10 |
|---|---|---|---|---|
| 00 | 1 | 0 | 0 | 0 |
| 01 | X | X | X | X |
| 11 | X | X | X | X |
| 10 | 0 | 0 | 1 | 0 |

$J_A = \overline{UD}\ \overline{QB}\ \overline{QC} + UD\ QB\ QC$

**For $J_B$**

| UD QA \ QB QC | 00 | 01 | 11 | 10 |
|---|---|---|---|---|
| 00 | 1 | 0 | X | X |
| 01 | 1 | 0 | X | X |
| 11 | 0 | 1 | X | X |
| 10 | 0 | 1 | X | X |

$J_B = \overline{(UD \oplus QC)}$

**For $K_A$**

| UD QA \ QB QC | 00 | 01 | 11 | 10 |
|---|---|---|---|---|
| 00 | X | X | X | X |
| 01 | 1 | 0 | 0 | 0 |
| 11 | 0 | 0 | 1 | 0 |
| 10 | X | X | X | X |

$K_A = \overline{UD}\ \overline{QB}\ \overline{QC} + UD\ QB\ QC$

**For $K_B$**

| UD QA \ QB QC | 00 | 01 | 11 | 10 |
|---|---|---|---|---|
| 00 | X | X | 0 | 1 |
| 01 | X | X | 0 | 1 |
| 11 | X | X | 1 | 0 |
| 10 | X | X | 1 | 0 |

$K_B = \overline{(UD \oplus QC)}$

**For $J_C$**

| UD QA \ QB QC | 00 | 01 | 11 | 10 |
|---|---|---|---|---|
| 00 | 1 | X | X | 1 |
| 01 | 1 | X | X | 1 |
| 11 | 1 | X | X | 1 |
| 10 | 1 | X | X | 1 |

$J_C = 1$

**For $K_C$**

| UD QA \ QB QC | 00 | 01 | 11 | 10 |
|---|---|---|---|---|
| 00 | X | 1 | 1 | X |
| 01 | X | 1 | 1 | X |
| 11 | X | 1 | 1 | X |
| 10 | X | 1 | 1 | X |

$K_C = 1$

**LOGIC DIAGRAM:**

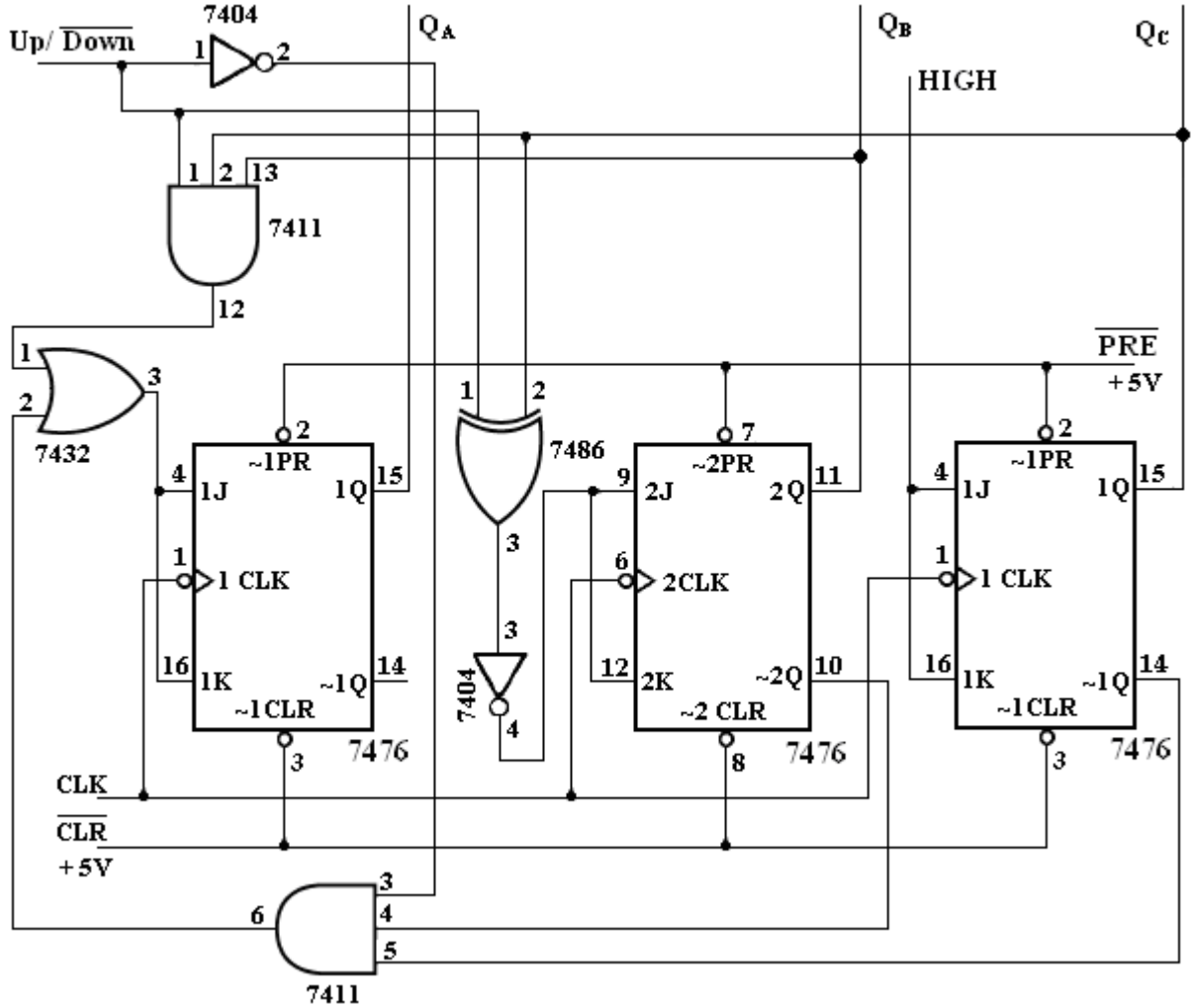



Figure 5.2 Logic Diagram of 3 bit synchronous up/down counter

**RESULT:**

Thus 3- bit synchronous up/down counter was designed and implemented successfully.

**4- BIT RIPPLE COUNTER:**

**PIN DIAGRAM: (JK Flip-Flop)**

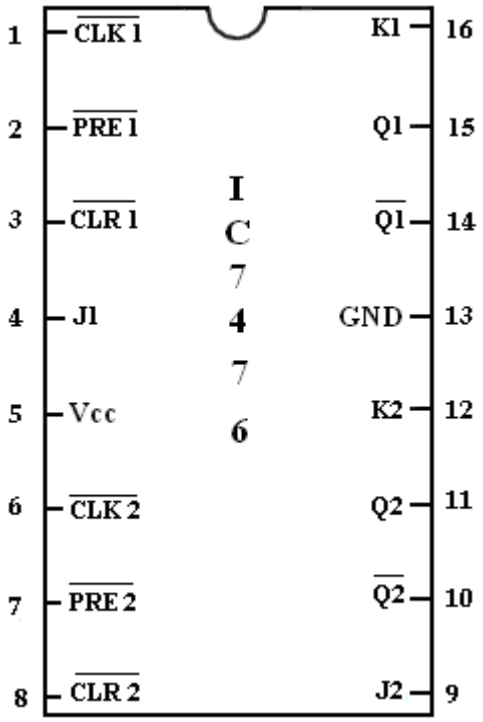



Figure 5.3 Pin Diagram of 4 bit ripple counter

**FUNCTION TABLE FOR 7476:**

Table 5.3 Function Table of 4 bit ripple counter

| Inputs | | | | | Outputs | |
|---|---|---|---|---|---|---|
| **Preset** | **Clear** | **Clock** | **J** | **K** | **Q** | **Q'** |
| 0 | 1 | X | X | X | 1 | 0 |
| 1 | 0 | X | X | X | 0 | 1 |
| 0 | 0 | X | X | X | 1 | 1 |
| 1 | 1 | ↓ | 0 | 0 | No Change | |
| 1 | 1 | ↓ | 0 | 1 | 0 | 1 |
| 1 | 1 | ↓ | 1 | 0 | 1 | 0 |
| 1 | 1 | ↓ | 1 | 1 | Toggle | |

**Ex. No. 7 : DESIGN AND IMPLEMENTATION OF 4-BIT RIPPLE COUNTER & MOD-10/MOD-12 RIPPLE COUNTERS**

**Date :**

**AIM:**

To design and implement 4 bit ripple counter, MOD-10 and MOD-12 ripple counter.

**APPARATUS REQUIRED:**

| Sl.No | Components | Specification | Quantity |
|---|---|---|---|
| 1. | NI- Multisim | Version 14.1 | - |
| 2. | PCS | Windows 10 | 1 |

**THEORY:**

- A counter is a register capable of counting number of clock pulse arriving at its clock input. Counter represents the number of clock pulse arrived. A specified sequence of states appears as counter output.
- This is the main difference between a register and a counter. There are two types of counter, synchronous and asynchronous.
- In synchronous common clock is given to all flip flop and in asynchronous, first flip flop is clocked by external pulse and then each successive flip flop is clocked by Q or Q' output of pervious stage.
- A *ripple counter* is a cascaded arrangement of flip-flops where the output of one flip-flop drives the clock input of the following flip-flop.
- The number of flip-flops in the cascaded arrangement depends upon the number of different logic states that it goes through before it repeats the sequence, a parameter known as the Modulus of the counter.
- In a ripple counter, also called an *asynchronous counter* or a *serial counter*, the clock input is applied only to the first flip-flop, also called the input flip-flop, in the cascaded arrangement.
- The clock input to any subsequent flip-flop comes from the output of its immediately preceding flip-flop.
- For instance, the output of the first flip-flop acts as the clock input to the second flip-flop, the output of the second flip-flop feeds the clock input of the third flip-flop and so on.

**LOGIC DIAGRAM: (4-Bit Ripple Counter)**

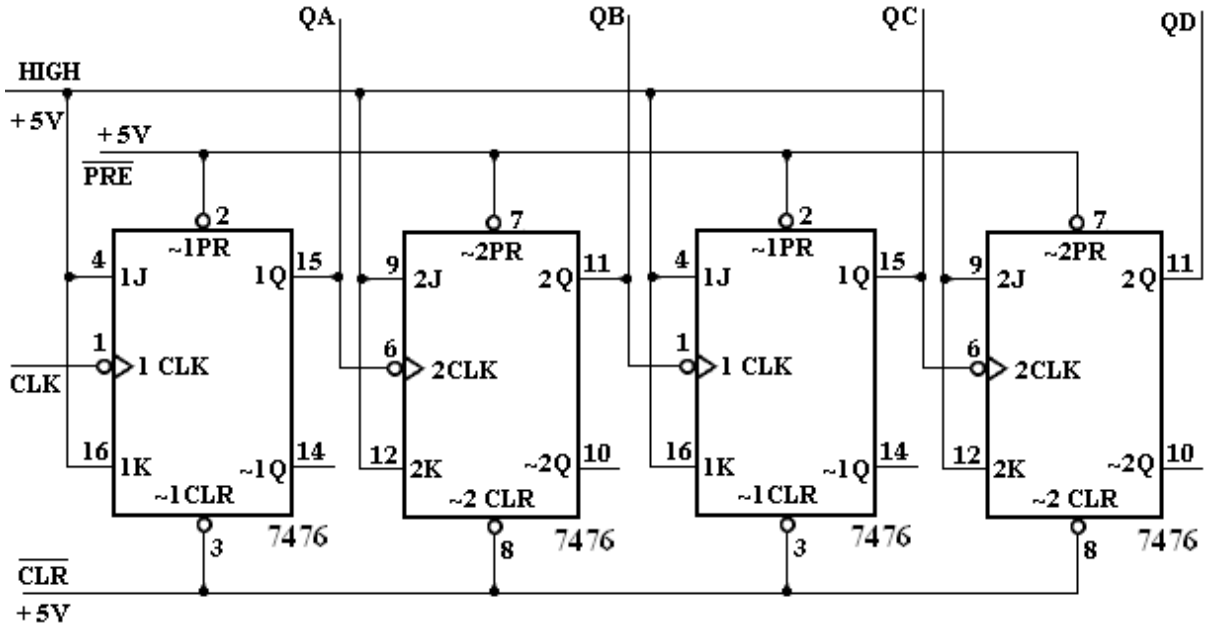



Figure 5.3 Logic Diagram of 4-Bit Ripple Counter

**TRUTH TABLE:**

Table 5.4 Truth Table of 4-Bit ripple counter

| CLK | $Q_A$ | $Q_B$ | $Q_C$ | $Q_D$ |
|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 |
| 1 | 1 | 0 | 0 | 0 |
| 2 | 0 | 1 | 0 | 0 |
| 3 | 1 | 1 | 0 | 0 |
| 4 | 0 | 0 | 1 | 0 |
| 5 | 1 | 0 | 1 | 0 |
| 6 | 0 | 1 | 1 | 0 |
| 7 | 1 | 1 | 1 | 0 |
| 8 | 0 | 0 | 0 | 1 |
| 9 | 1 | 0 | 0 | 1 |
| 10 | 0 | 1 | 0 | 1 |
| 11 | 1 | 1 | 0 | 1 |
| 12 | 0 | 0 | 1 | 1 |
| 13 | 1 | 0 | 1 | 1 |
| 14 | 0 | 1 | 1 | 1 |
| 15 | 1 | 1 | 1 | 1 |

- A four-bit ripple counter is implemented with negative edge-triggered *J-K* flip-flops wired as toggle flip-flops.
- The output of the first flip-flop feeds the clock input of the second, and the output of the second flip-flop feeds the clock input of the third, the output of which in turn feeds the clock input of the fourth flip-flop.
- The outputs of the four flip-flops are designated as Q0 (LSB flip-flop), Q1, Q2 and Q3 (MSB flip-flop).

**LOGIC DIAGRAM: (MOD-10 Ripple Counter)**

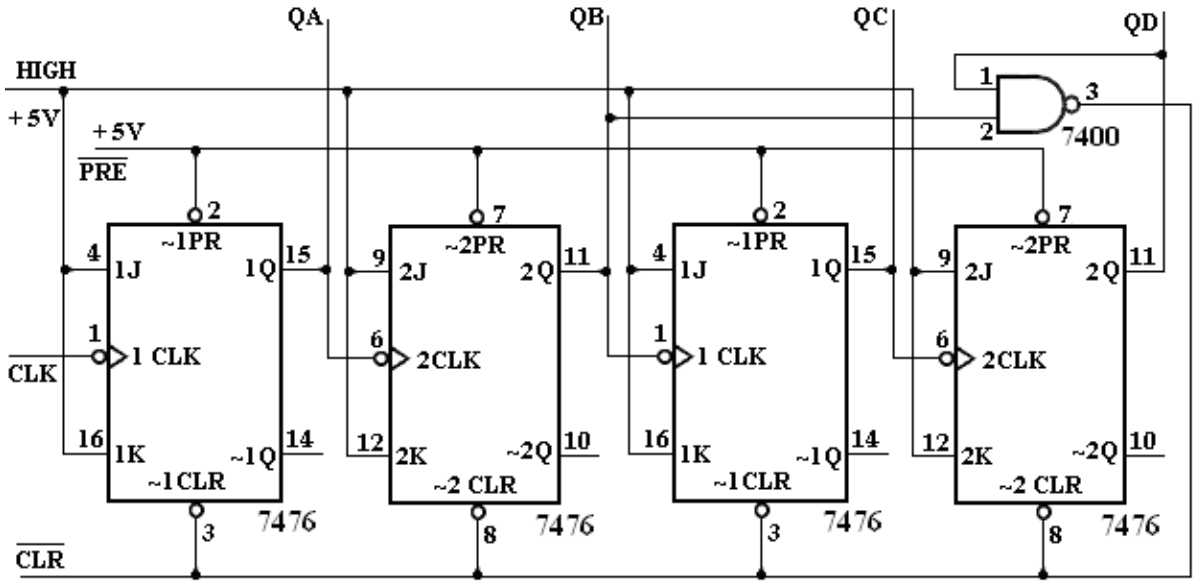



Figure 5.4 Logic Diagram of MOD-10 Ripple Counter

**TRUTH TABLE**:

Table 5.5 Truth Table of MOD-10 Ripple Counter

| CLK | $Q_A$ | $Q_B$ | $Q_C$ | $Q_D$ |
|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 |
| 1 | 1 | 0 | 0 | 0 |
| 2 | 0 | 1 | 0 | 0 |
| 3 | 1 | 1 | 0 | 0 |
| 4 | 0 | 0 | 1 | 0 |
| 5 | 1 | 0 | 1 | 0 |
| 6 | 0 | 1 | 1 | 0 |
| 7 | 1 | 1 | 1 | 0 |
| 8 | 0 | 0 | 0 | 1 |
| 9 | 1 | 0 | 0 | 1 |
| 10 | 0 | 0 | 0 | 0 |

**LOGIC DIAGRAM: (MOD-12 Ripple Counter)**

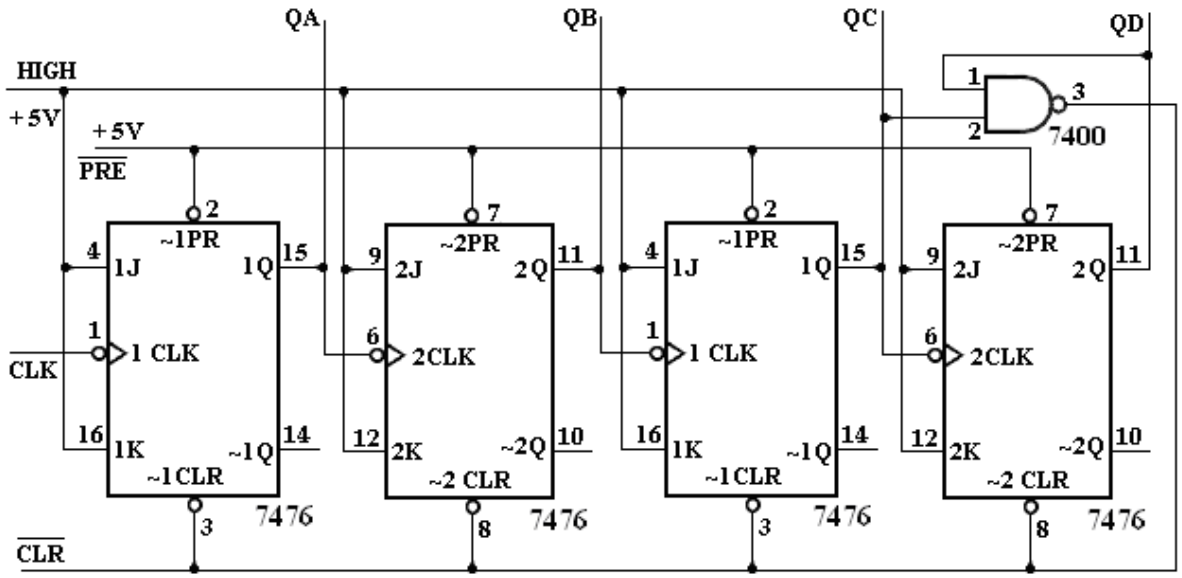



Figure 5.5 Logic Diagram of MOD-12 Ripple Counter

**TRUTH TABLE**:

Table 5.6 Truth Table of MOD-12 Ripple Counter

| CLK | $Q_A$ | $Q_B$ | $Q_C$ | $Q_D$ |
|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 |
| 1 | 1 | 0 | 0 | 0 |
| 2 | 0 | 1 | 0 | 0 |
| 3 | 1 | 1 | 0 | 0 |
| 4 | 0 | 0 | 1 | 0 |
| 5 | 1 | 0 | 1 | 0 |
| 6 | 0 | 1 | 1 | 0 |
| 7 | 1 | 1 | 1 | 0 |
| 8 | 0 | 0 | 0 | 1 |
| 9 | 1 | 0 | 0 | 1 |
| 10 | 0 | 1 | 0 | 1 |
| 11 | 1 | 1 | 0 | 1 |
| 12 | 0 | 0 | 0 | 0 |

**PROCEDURE:**

1. Connections are given as per the logic diagram.
2. Logic inputs are given as per the logic diagram.
3. Observe the logic output and verify with the truth tables.

**RESULT:**

Thus 4-bit ripple counter, MOD-10 and MOD-12 ripple counter was designed and implemented successfully.

## i. SHIFT REGISTER:

**PIN DIAGRAM: (D-Flip-Flop)**

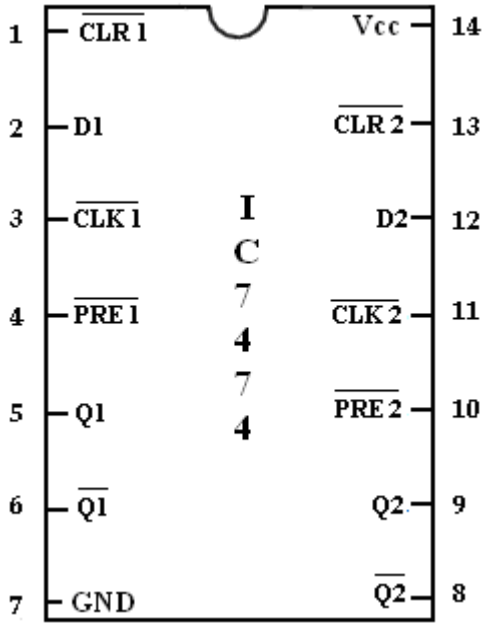



Figure 6.1 Pin Diagram of D-Flip-Flop

**FUNCTION TABLE:**

Table 6.1 Function Table of D-Flip-Flop

| Inputs | | | | Outputs | |
|---|---|---|---|---|---|
| **Preset** | **Clear** | **Clock** | **D** | **Q** | **Q'** |
| 0 | 1 | X | X | 1 | 0 |
| 1 | 0 | X | X | 0 | 1 |
| 0 | 0 | X | X | 1 | 1 |
| 1 | 1 | ↑ | 0 | 0 | 1 |
| 1 | 1 | ↑ | 1 | 1 | 0 |
| 1 | 1 | 0 | X | No Change | |

**Ex. No. 6 : DESIGN AND IMPLEMENTATION OF SHIFT REGISTERS**
**Date :**

**AIM:**

To design and implement

1. Serial in serial Out(SISO)
2. Serial in parallel Out(SIPO)
3. Parallel in serial Out(PISO)
4. Parallel in parallel Out(PIPO)

**APPARATUS REQUIRED:**

| Sl.No | Components | Specification | Quantity |
|---|---|---|---|
| 1. | NI- Multisim | Version 14.1 | - |
| 2. | PCS | Windows 10 | 1 |

**THEORY:**

- A register is capable of shifting its binary information in one or both directions is known as shift register.
- A logical configuration of shift register consist of a D flip flop cascaded with output of one flip flop connected to input of next flip flop.
- All flip flops receive common clock pulses which causes the shift in the output of the flip flop. The simplest possible shift register is one that uses only flip flop.
- The output of a given flip flop is connected to the input of next flip flop of the register.
- Each clock pulse shifts the content of register one bit position to right.

## ii. SERIAL IN SERIAL OUT:

### LOGIC DIAGRAM:

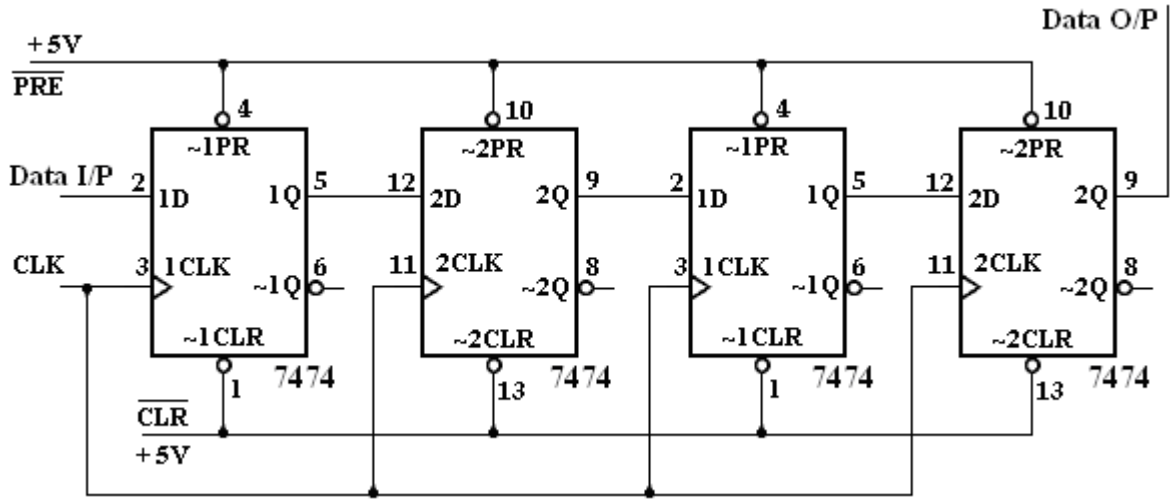



Figure 6.2 Logic Diagram of Serial In Serial Out

### TRUTH TABLE:

Table 6.2 Truth Table of Serial In Serial Out

| CLK | Serial IN | Serial OUT |
|---|---|---|
| 1 | 1 | 0 |
| 2 | 1 | 0 |
| 3 | 1 | 0 |
| 4 | 1 | 1 |
| 5 | 0 | 1 |
| 6 | 0 | 1 |
| 7 | 0 | 1 |
| 8 | 0 | 0 |

## iii. SERIAL IN PARALLEL OUT:

### LOGIC DIAGRAM:

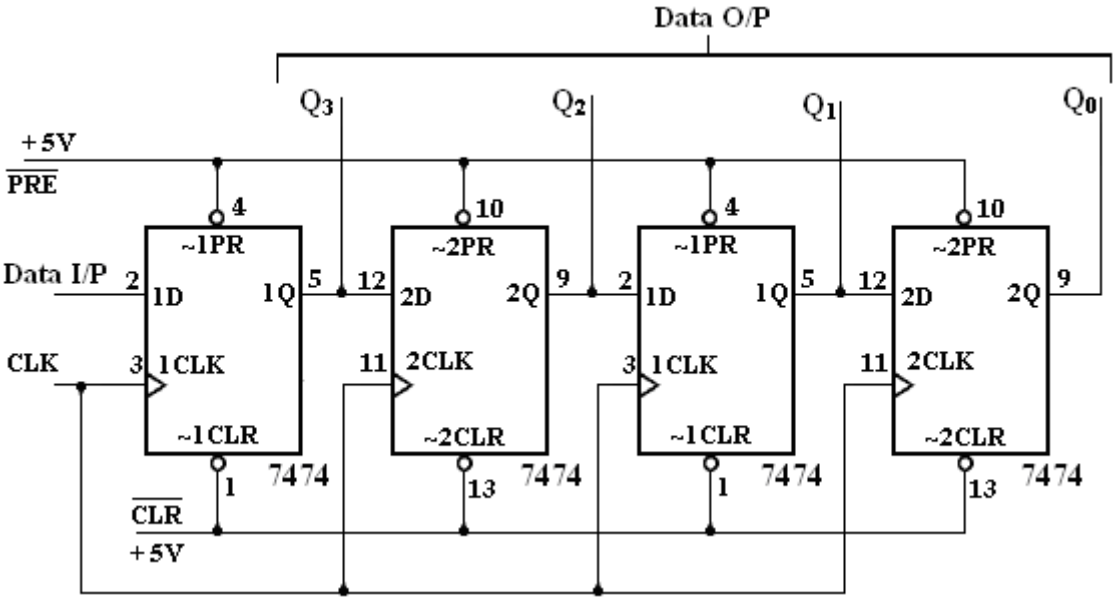



Figure 6.3 Logic Diagram of Serial In Parallel Out

### TRUTH TABLE:

Table 6.3 Truth Table of Serial In Parallel Out

| CLK | DATA | OUTPUT | | | |
|---|---|---|---|---|---|
| | | $Q_3$ | $Q_2$ | $Q_1$ | $Q_0$ |
| 1 | 1 | 1 | 0 | 0 | 0 |
| 2 | 0 | 0 | 1 | 0 | 0 |
| 3 | 0 | 0 | 0 | 1 | 0 |
| 4 | 1 | 1 | 0 | 0 | 1 |

## iv. PARALLEL IN SERIAL OUT:

### LOGIC DIAGRAM:

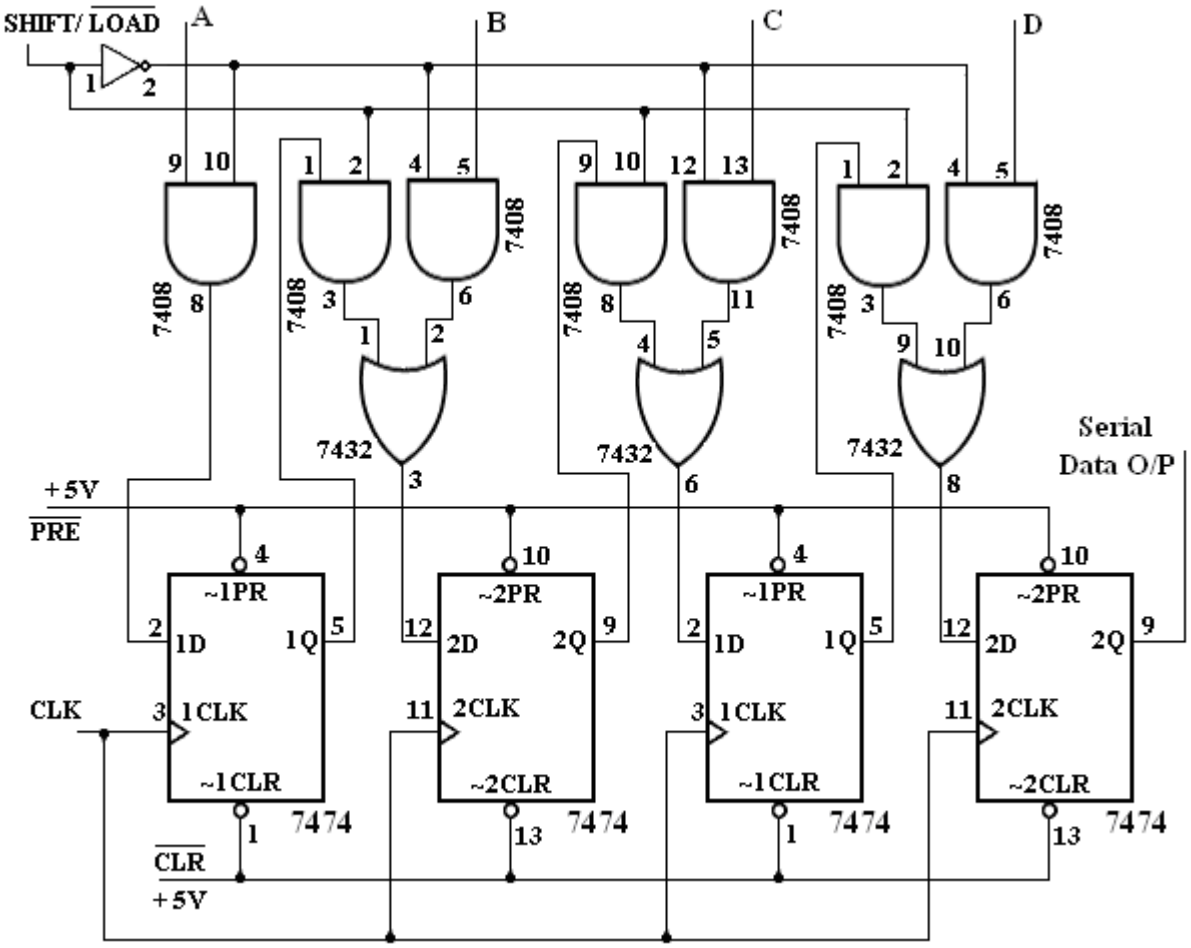



Figure 6.4 Logic Diagram of Parallel In Serial Out

### TRUTH TABLE:

Table 6.4 Truth Table of Parallel In Serial Out

| S/ L' | CLK | INPUTS | | | | OUTPUT |
|---|---|---|---|---|---|---|
| | | A | B | C | D | Q |
| 0 | 0 | 1 | 0 | 0 | 1 | 1 |
| 1 | 1 | 1 | 0 | 0 | 1 | 0 |
| 1 | 2 | 1 | 0 | 0 | 1 | 0 |
| 1 | 3 | 1 | 0 | 0 | 1 | 1 |

## v. PARALLEL IN PARALLEL OUT:

### LOGIC DIAGRAM:

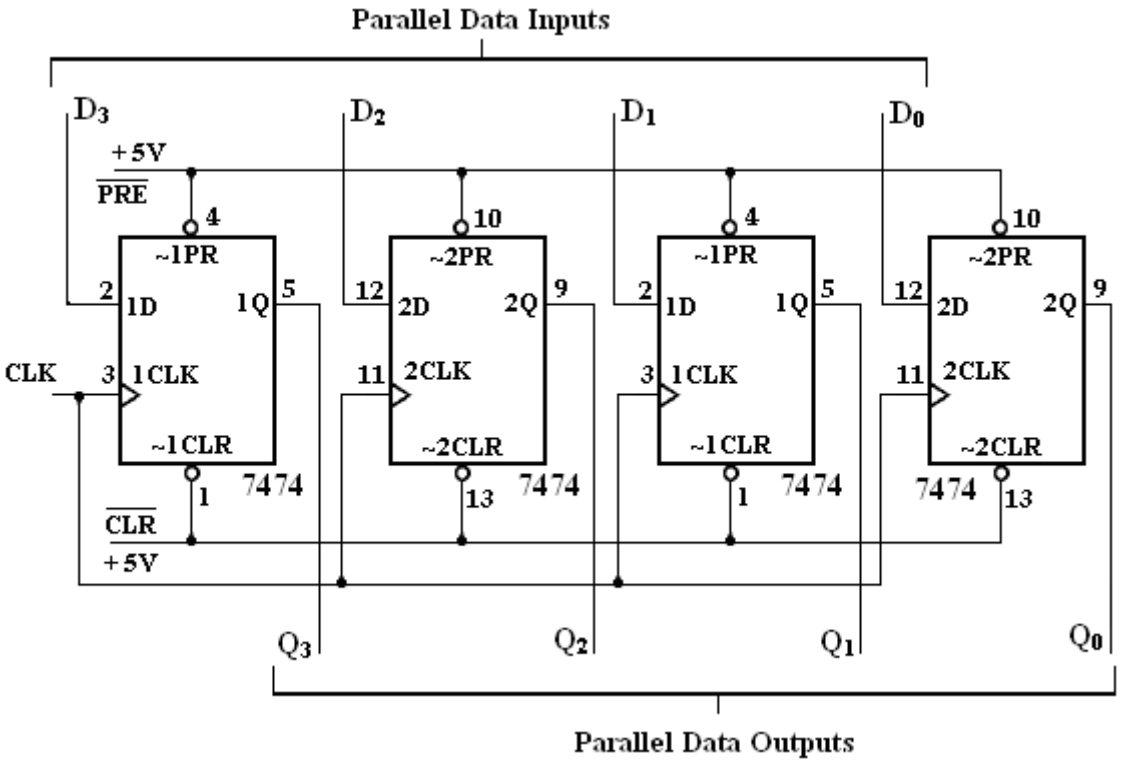



Figure 6.5 Logic Diagram of Parallel In Parallel Out

### TRUTH TABLE:

Table 6.5 Truth Table of Parallel In Parallel Out

| CLK | DATA INPUTS | | | | OUTPUT | | | |
|---|---|---|---|---|---|---|---|---|
| | $D_3$ | $D_2$ | $D_1$ | $D_0$ | $Q_3$ | $Q_2$ | $Q_1$ | $Q_0$ |
| 1 | 1 | 0 | 0 | 1 | 1 | 0 | 0 | 1 |
| 2 | 1 | 0 | 1 | 0 | 1 | 0 | 1 | 0 |

**PROCEDURE:**

1. Connections are given as per the logic diagram.
2. Logic inputs are given as per the logic diagram.
3. Observe the logic output and verify with the truth tables.

**RESULT:**

Thus the design and implementation of

1. Serial in serial Out (SISO)
2. Serial in parallel Out (SIPO)
3. Parallel in serial Out (PISO)
4. Parallel in parallel Out (PIPO) were done successfully.